Lesson No. 4. The pandas Package

Lab 3. The pandas Package

Tasks Part I

Read the text "User Guide '10 minutes to pandas'" (lecture text) or `https://pandas.pydata.org/docs/user_guide/10min.html` and execute the commands from it in Jupyter/Google Colab (see also the pandas documentation and books from the reference list; this part does not need to be submitted).

Part II

Use the methods of the pandas package. See the group list for task numbers.

Creating a Series Object (one-dimensional sequence with an index)

1. Create a Series object from a list.
2. Create a Series object from a list. When creating it, specify an index explicitly, also as a list.
3. Create a Series object from a dictionary.
4. Create a Series object from a generated random numpy array.
5. Display the representation of the array data (values) and its index (index). Access an element of the Series by index and change its value. Assign a name to the Series object (the name property), and to its index (index.name). Output its value type (dtype).

Creating a DataFrame Object

(a table with column headers and a row index)

6. Create a DataFrame object from a random numpy matrix. When creating it, specify an index (using the index attribute), for example, assign a list of numbers.
7. Create a DataFrame object from a random numpy matrix. When creating it, specify column names (using the columns attribute), for example, assign a list of names.
8. Create a DataFrame object from a dictionary of lists of equal length. Use data in different columns of different types. For example, in the first column – integers, in the second – strings, etc.
9. Create a DataFrame object from a dictionary of Series objects. Use data in different Series objects of different types. For example, in the first – integers, in the second – strings, etc.
10. Display the representation of the data as a numpy matrix (values), its index (index), columns (columns), and column types (dtypes).
11. Display information about the DataFrame object created above: dimensionality (ndim, shape), number of elements (size).
12. Access a single column of the DataFrame object: name[column name]. Display the representation of the data of this column (values) and its index (index). Output its name (name) and its value type (dtype).

Changing the Index of an Object

(pandas.Index: `https://pandas.pydata.org/docs/reference/api/pandas.Index.html`)

13. Create a new index object (pd.Index) and use it when creating a new DataFrame table, setting the index property.
14. Create a new index object (pd.Index) and use it to change the index of an existing DataFrame table by assigning it to the index property.

    (reindex: `https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.reindex.html`)

15. Change the index of an existing DataFrame table: change the order. To do this, create a new index object (pd.Index) with a changed order of elements in the index and use it in the reindex method of the DataFrame object.
16. Change the index of an existing DataFrame table: add new elements to it. To do this, create a new index object (pd.Index) and use it in the reindex method of the DataFrame object.
17. Change the index of an existing DataFrame table: add new elements to it. To do this, create a new index object (pd.Index) and use it in the reindex method of the DataFrame object (use its fill_value parameter).
18. Change the order of columns using the reindex method.
19. Add a new column to the DataFrame object using assignment: "name[new column name] = value".
20. Delete a column from the DataFrame object using del: "del name[column name to delete]".

Viewing Data

21. Output the first 3 rows of the DataFrame object (head method).
22. Output the last 3 rows of the DataFrame object (tail method).
23. Display a statistical summary for the DataFrame object (describe).
24. Display information about the DataFrame object (info).

Selections

25. Select a column using access by column name.
26. Select several columns using access by column names.
27. Select several rows using slices by row indices.
28. Select several rows using slices by row numbers.
29. Select a row using a label (loc method).
30. Select simultaneously part of rows and columns using labels (loc method).
31. Select a single value using labels (loc method).
32. Select a single value using labels (at method).
33. Select simultaneously part of rows and columns using indices (iloc method).
34. Select a single value using indices (iloc method).
35. Select a single value using indices (iat method).

Boolean Indexing

36. Using a boolean condition, select rows that satisfy some condition.
37. Using a boolean condition, select elements that satisfy some condition.
38. Using the isin method, select rows that satisfy some condition.

Setting Values

39. Set a single value using labels (at).
40. Set a single value using indices (iat).
41. Set several values at once in a certain slice (loc).
42. Set values only for elements that satisfy some condition.

Missing Data

43. Delete all rows where all values are undefined (dropna method).
44. Delete all columns that have undefined values (dropna method).
45. Fill missing values in one column with zeros (fillna method).
46. Fill missing values in one column with values from previous elements (fillna method, method='fill' parameter).
47. Fill missing values in different columns with different values (fillna method, value = dictionary parameter).
48. Select all rows in the DataFrame where the value in the first column is not set (isna method).
49. Select all rows in the DataFrame where the value in the first column is not missing (notna method).
50. Check for duplicates in the DataFrame (duplicated).
51. Check for duplicates in one column of the DataFrame (duplicated).

Operations

52. Increase all values in one column of the DataFrame by 10 times.
53. Subtract the minimum value of that column from all elements in one column of the DataFrame.
54. Multiply all elements in one column of the DataFrame that are less than 100 in absolute value by 10.
55. Increase all elements in one column of the DataFrame that are less than 10 by 10.

Descriptive Statistics

56. Sum, mean, and minimum values.
57. Product, standard deviation, and index of the minimum value.
58. Mean, variance, and median.

Methods unique, value_counts, sort_values, sort_index

59. Series.unique method.
60. DataFrame.value_counts method.
61. DataFrame.value_counts(dropna=False) method.
62. DataFrame.sort_values method.
63. DataFrame.sort_index method.

Methods apply, map, where, sort_values

(In this section, using the documentation/books, figure out what the method is used for; apply it.)

64. DataFrame.apply method.
65. DataFrame.map method.
66. DataFrame.replace method.
67. DataFrame.where method.

Methods groupby, pivot_table

(In this section, using the documentation/books, figure out what the method is used for; apply it.)

68. DataFrame.groupby method.
69. DataFrame.pivot_table method.

Data Normalization

70. Apply decimal scaling to the data in each column of the DataFrame. To do this, for each column find the maximum absolute value. And divide all elements by 10^n, where n is such that the found $10^{n-1} \leq \max \leq 10^n$, i.e., if $\max = 456$, then divide by 1000.
71. Apply min-max normalization to one column of the DataFrame. To do this, transform all elements x of the column according to the formula: $x = \frac{x - \min}{\max - \min}$, where $\min, \max$ are the minimum and maximum in this column.
72. Apply min-max normalization to one column of the DataFrame. To do this, transform all elements x of the column according to the formula: $x = \frac{2(x - \min)}{\max - \min} - 1$, where $\min, \max$ are the minimum and maximum in this column.
73. Apply mean normalization to one column of the DataFrame. To do this, transform all elements x of the column according to the formula: $x = \frac{x - \text{mean}}{\text{var}}$, where mean, var are the mean and variance in this column.

Data Input/Output to File

`https://pandas.pydata.org/docs/reference/io.html`
`pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html`
`https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html`

74. Write the DataFrame to a csv file (to_csv method).
75. Read a DataFrame object from a csv file (pandas.read_csv function).

References

1. Documentation `https://pandas.pydata.org/docs/index.html`